Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 12

Roll No : 459

Name : Vibhu Shihora

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

In [2]:
```
#ZeroDivision Error

try:
    ans = 10 / 0

except ZeroDivisionError as msg:
    print(msg)


# Value Error

try:
    b = int("abc")

except ValueError as msg:
    print(msg)

# Type Error
```

```
try:
    print(10 +"90")

except TypeError as msg:
    print(msg)
```

```
division by zero
invalid literal for int() with base 10: 'abc'
unsupported operand type(s) for +: 'int' and 'str'
```

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

In [7]:
```
#Index Error

try :
    li = [1,2,3]
    print(li[10])
except IndexError as msg:
    print(msg)


#Key Error

try:
    di = {1 : "vi" , 2 : "jay"}
    print(di[10])

except KeyError as msg:
    print("Error : " , msg)
```

```
list index out of range
Error :  10
```

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

In [8]:
```
#FileNotFound Error

try:
    fp = open("ok.txt","r")

except FileNotFoundError as msg :
    print(msg)

#ModuleNotFound Error

try:
    import module

except ModuleNotFoundError as msg :
    print(msg)
```

```
[Errno 2] No such file or directory: 'ok.txt'
No module named 'module'
```

04) WAP that handles all type of exceptions in a single except block in standard error message format.

In [9]:
```python
# Handle All Exception

try:
    import module

except Exception as msg :
    print(msg)
```

```
No module named 'module'
```

05) WAP to demonstrate else and finally block.

In [11]:
```python
try:
    print(10 + 0)
except Exception as msg:
    print(msg)
else:
    print("No Exception Found")
finally:
    print("Program Completed")
```

```
10
No Exception Found
Program Completed
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

In [18]:
```python
def divide(a,b):

    try:
        ans = a / b
    except ZeroDivisionError as msg :
        print(msg)
    else:
        return ans

print(divide(10,0))
```

```
division by zero
None
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

In [30]:
```python
class NegativeValue(Exception):
    pass

try:
    li = []
```

```
    total = 0

    for i in range(1,5):
        a = int(input("Enter Element : "))

        if(a < 0):
            raise NegativeValue("Negative Mark")

        li.append(a)
        total += a
except Exception as msg:
    print(msg)
else:
    print(total)
finally:
    print(li)
```

```
Negative Mark
[99]
```

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

In [33]:
```
try:
    s = input("Enter Grade sep by (,) : ").split(",")
    s = [int(i) for i in s]

except Exception as msg:
    print(msg)
```

```
invalid literal for int() with base 10: 'h'
```

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

In [38]:
```
try:
    li = []
    total = 0
```

```
    for i in range(1,6):
        a = int(input("Enter Markd : "))

        assert 0 <= a <= 100 , "Marks should be between 0 and 100."

        li.append(a)
        total += a

except Exception as msg:
    print(msg)

finally:
    print("li : ",li)
    print("Total : " , total / len(li))
```

```
Marks should be between 0 and 100.
li :  [88, 90]
89.0
```

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the messsage "Correct" otherwise.

In [50]:
```
class WeakPasswordError(Exception):
    pass

try:
    pas = input("Enter PassWord : ")

    countDig = 0

    for i in pas:
        if i.isdigit():
            countDig += 1
    print(countDig)

    if(len(pas) < 8 or countDig != 1):
        raise WeakPasswordError("Invalid")

except Exception as err:
    print(err)
else:
    print("Ok")
```

```
1
Ok
```

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

In [54]:
```
class NegativeValue(Exception):
    pass

try:
    a = int(input("Enter Element : "))

    if(a < 0):
        raise NegativeValue("Negative")

    li.append(a)
    total += a
except Exception as msg:
    print(msg)
else:
    print(pow(a,0.5))
finally:
    print("Complete Program")
```

```
Negative
Complete Program
```